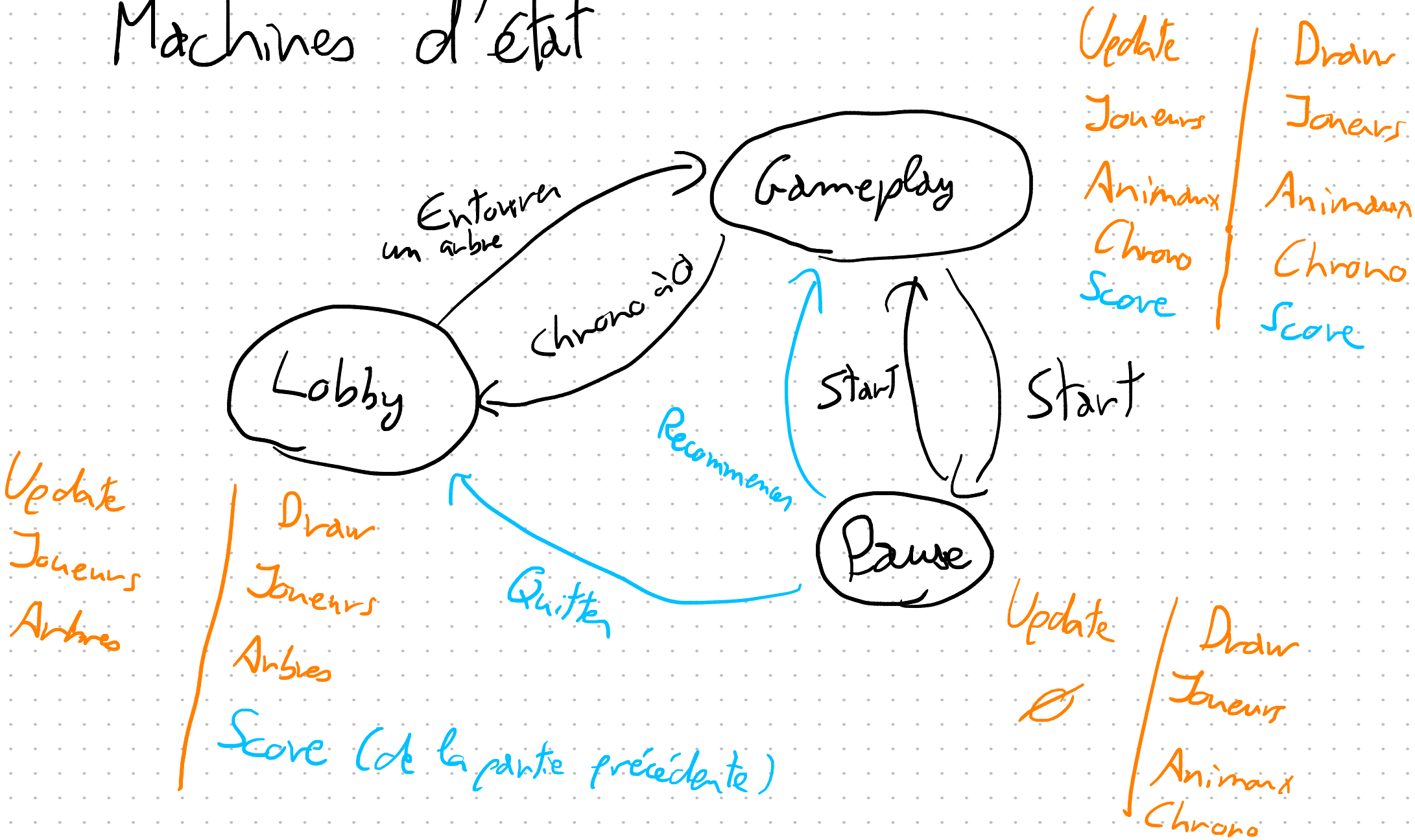
État du jeu est représenté par deux booléens :

m_bPause \ m_bLobby	FAUX	VRAI
FAUX	GAME PLAY	LOBBY
VRAI	PAUSE	∅

Ne doit pas arriver

Combiner N booléens mènent à 2^N états possibles => Explosion Combinatoire

Machines d'état



1ère manière | Faire un enum

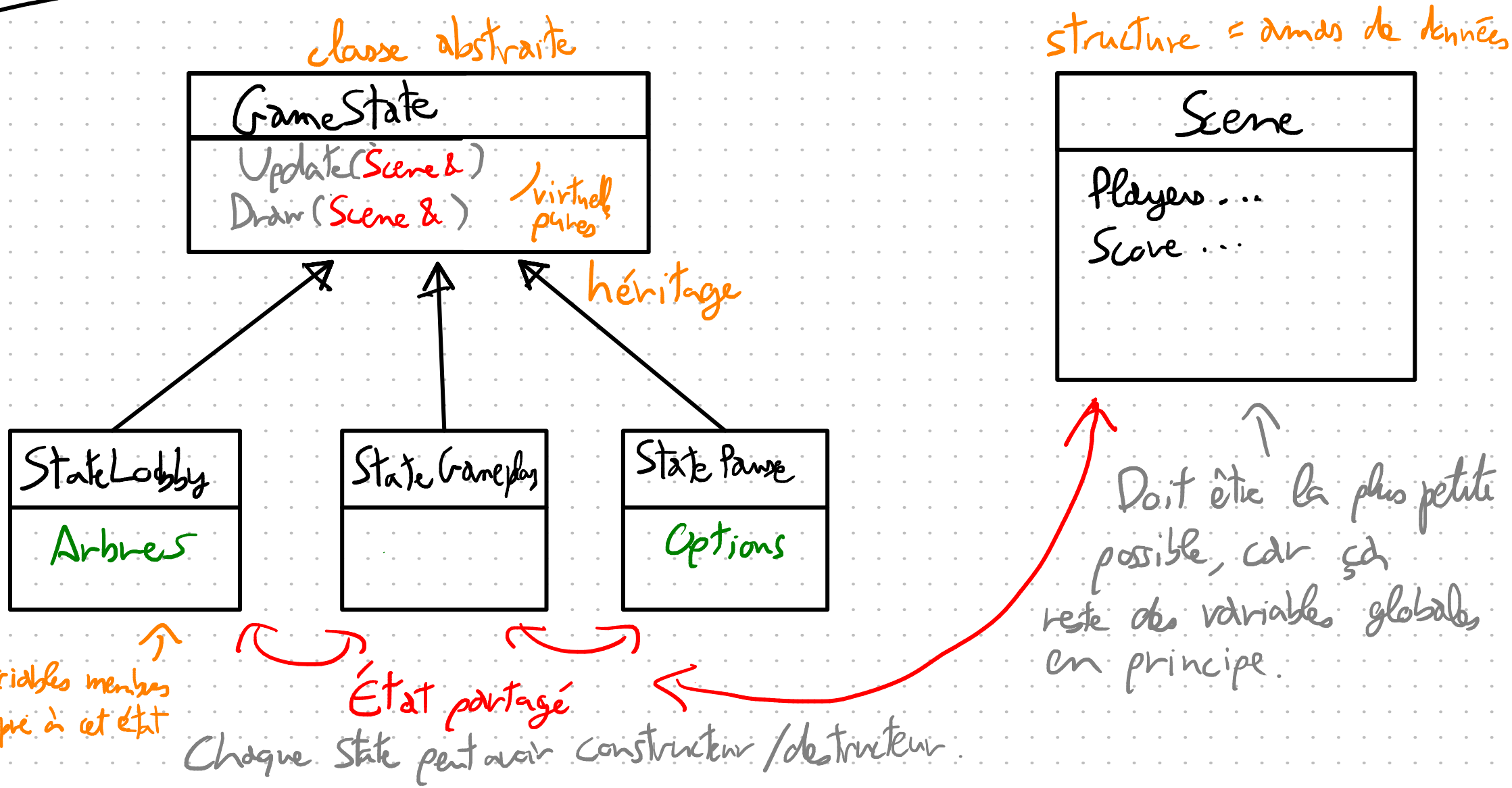
```
enum class EState { Lobby, Gameplay, Pause };

switch (g_eState) {
case EState::Lobby:    UpdateLobby();  break;
case EState::Gameplay: UpdateGameplay(); break;
case EState::Pause:    UpdatePause();  break;
}
```

→ Update

→ Draw

→ Transitions d'états

2ème manière | Programmation orientée objet



Variables globales :

```
Scene g_Scene;
GameState* g_pState;
```

Données pures et commune

Logique actuelle

```
Dino_GameInit() {
    g_pState = new StateLobby(——);
}
```

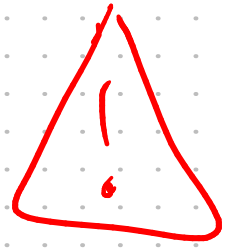
Allocation dynamique

constructeur

```
Dino_GameFrame() {
    GameState* pNext = g_pState->Update(g_Scene, time, deltatime...);
    if (pNext != nullptr) {
        delete g_pState;
        g_pState = pNext;
    }
}
```

Destruction + Désallocation

+Shut() : delete g_pState;



```
GameState * p = new StateLobby();
p -> Update(—);
```

virtuel parce que c'est marqué dans GameState

```
delete p;
```

↳ Appelle le destructor ~GameState() non-virtuel

→ Les ressources des classes dérivées ne sont pas libérés.

⟹ Marquer le destructeur de GameState comme virtuel.

```
virtual ~GameState() = 0;
                     = default;
```